# Doma Assist

Aplicativo Wear OS de assistência por áudio

Trabalho Prático | DGT2816 - Interação com sensores de smartphones e wearables
Aluno: Daniel Victor Reis | Data: 22/10/2026

## 1. Introdução

O enunciado do trabalho apresenta a empresa Doma, que quer um aplicativo de relógio para ajudar funcionários com necessidades especiais. A proposta é usar o áudio para dar informações, orientações e alertas de segurança, o que pode ser bem útil, por exemplo, para quem tem deficiência visual.

Com isso em mente, fiz o **Doma Assist**, um app para Wear OS que fala avisos em português, descobre quais saídas de áudio o relógio tem (alto-falante e Bluetooth), percebe quando um fone é conectado ou desconectado e toca um arquivo de som. Segui bastante o roteiro de prática da disciplina, principalmente a parte de saídas de áudio.

## 2. Ambiente de desenvolvimento

- Android Studio, com o projeto criado para Wear OS.
- Emulador Wear OS criado no Device Manager, com o nome **Doma API 37.0**.
- Linguagem Kotlin e interface feita com Jetpack Compose para Wear OS (Material 3).
- Pacote do projeto: `com.doma.assist`.

## 3. O que o app faz

A tela principal tem um texto de status no topo e uma lista de botões. O texto de status muda conforme o que está acontecendo (iniciando a voz, reproduzindo um aviso, fone conectado e assim por diante).

| Botão | Função |
|---|---|
| Ouvir aviso | O relógio fala uma mensagem de boas-vindas. |
| Alerta de segurança | O relógio fala um aviso de emergência, orientando a procurar um local seguro. |
| Ajuda (botão da borda) | Fala uma frase curta de apoio. |
| Verificar áudio | Mostra se há alto-falante e/ou Bluetooth disponível. |
| Tocar áudio | Toca o arquivo de som que está dentro do app. |
| Conectar fone | Abre as configurações de Bluetooth do relógio para parear um fone. |

## 4. Como foi implementado

### 4.1 Descobrindo as saídas de áudio

Criei a classe `AudioHelper`, parecida com a do roteiro. Ela primeiro confere se o aparelho tem a característica `FEATURE_AUDIO_OUTPUT` e, se tiver, usa o `getDevices()` do `AudioManager` para listar as saídas. Daí dá para perguntar se existe alto-falante embutido (`TYPE_BUILTIN_SPEAKER`) ou fone Bluetooth.

```
fun audioOutputAvailable(type: Int): Boolean {
    if (!hasAudioOutputFeature()) return false

    return audioManager
        .getDevices(AudioManager.GET_DEVICES_OUTPUTS)
        .any { it.type == type }
}
```

Uma diferença em relação ao roteiro: para Bluetooth eu considerei tanto o `TYPE_BLUETOOTH_A2DP` (áudio de mídia) quanto o `TYPE_BLUETOOTH_SCO` (usado em chamadas e headsets), porque dependendo do fone ele pode aparecer de um jeito ou de outro.

### 4.2 Detectando fones conectados na hora

O roteiro pede um callback com `registerAudioDeviceCallback`. Coloquei isso dentro do `AudioHelper`. Sempre que um dispositivo de áudio é adicionado ou removido, o app confere de novo se há um fone Bluetooth e, se o estado mudou, atualiza o texto de status com "Fone Bluetooth conectado" ou "desconectado". O callback é registrado quando a tela abre e removido quando ela fecha, para não ficar vazando recurso.

```
val callback = object : AudioDeviceCallback() {
    override fun onAudioDevicesAdded(addedDevices: Array<out AudioDeviceInfo>?) {
        onDevicesChanged()
    }
    override fun onAudioDevicesRemoved(removedDevices: Array<out AudioDeviceInfo>?) {
        onDevicesChanged()
    }
}
audioManager.registerAudioDeviceCallback(callback, null)
```

### 4.3 Voz com TextToSpeech

Os avisos falados usam o `TextToSpeech` do Android, configurado para português do Brasil (pt-BR). O app só libera a fala depois que a voz termina de iniciar, e se o idioma não estiver instalado ele avisa no status em vez de ficar em silêncio. Também usei um `UtteranceProgressListener` para o status voltar para "Pronto para ajudar" quando a fala acaba.

### 4.4 Tocando um arquivo de áudio

Para o botão "Tocar áudio" usei o `MediaPlayer` com um arquivo colocado na pasta `res/raw` (o `aviso.wav`). Quando o som termina, o player é liberado, e ele também é liberado se a tela for fechada.

### 4.5 Abrindo o Bluetooth para o usuário

Seguindo o passo 4 do roteiro, o botão "Conectar fone" leva o usuário direto para as configurações de Bluetooth usando `Settings.ACTION_BLUETOOTH_SETTINGS`. No exemplo do roteiro aparece também a flag `FLAG_ACTIVITY_CLEAR_TASK` e alguns extras, mas eu não usei. A flag apagaria a pilha de telas do próprio app e os extras não são padrão do Android. Deixei só o `FLAG_ACTIVITY_NEW_TASK`, e coloquei um tratamento para o caso de o relógio não ter essa tela.

```
val intent = Intent(Settings.ACTION_BLUETOOTH_SETTINGS).apply {
    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
}
context.startActivity(intent)
```

### 4.6 AndroidManifest.xml

- `uses-feature android.hardware.type.watch`, porque o app é só para relógio.
- Permissão `WAKE_LOCK`, que o roteiro pede.
- Bloco `<queries>` com `TTS_SERVICE`, necessário no Android 11 ou superior para o app encontrar o mecanismo de voz.
- A `MainActivity` com `exported="true"` e o intent-filter MAIN/LAUNCHER.

## 5. Testes e evidências

Os testes foram feitos no emulador Wear OS do Android Studio. As capturas abaixo foram tiradas pela própria barra do emulador.

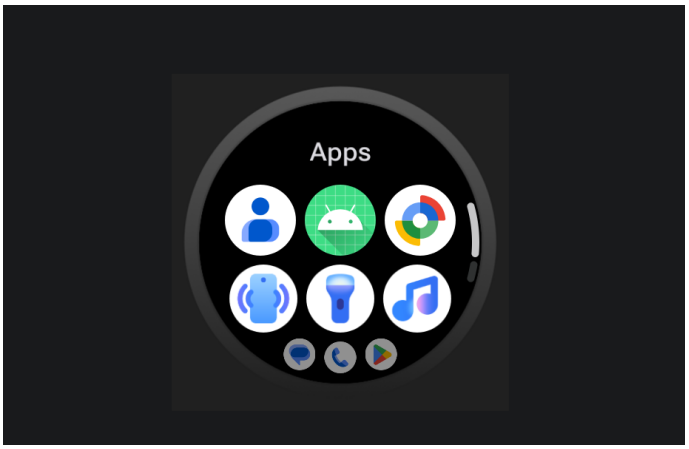



*Figura 1 - Lista de apps do relógio no emulador, onde o Doma Assist aparece com o ícone padrão do projeto (o robô do Android sobre fundo verde).*




*Figura 2 - Configurações do relógio. A opção Conectividade (Connectivity) leva às opções de Bluetooth e Wi-Fi.*

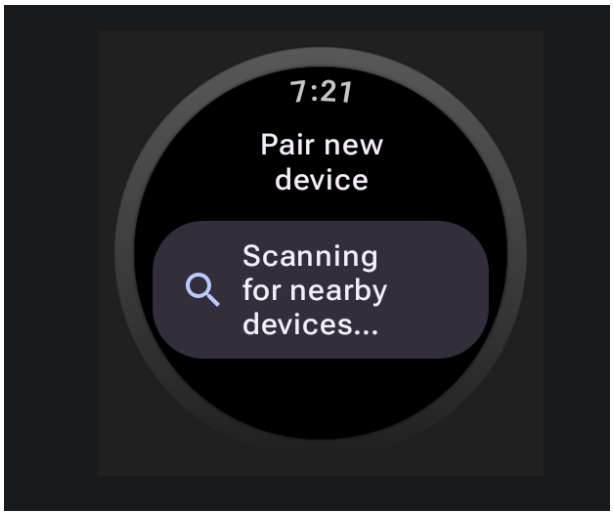



*Figura 3 - Tela "Pair new device" aberta pelo botão "Conectar fone" do app. Ela fica procurando dispositivos próximos.*




*Figura 4 - Configurações de Wi-Fi do emulador, conectado à rede virtual AndroidWifi.*

A Figura 3 mostra que o passo de levar o usuário às configurações de Bluetooth funciona: o botão do app abre a tela de pareamento do relógio. A busca, porém, não encontra nada. Isso acontece porque o emulador do Wear OS não tem Bluetooth de verdade, só simula o Wi-Fi (Figura 4). Então não foi possível parear um fone nem ver o aviso de "Fone Bluetooth conectado" aparecendo. Essa

parte está programada conforme o roteiro, mas só poderia ser comprovada em um relógio real com um fone.

## 6. Dificuldades que apareceram

- **Erro de compilação com R.raw.** O Android Studio acusou "Unresolved reference 'raw'" e mais alguns erros em sequência. A causa era simples: a pasta `res/raw` e o arquivo de áudio ainda não existiam no projeto. Depois que criei a pasta e coloquei o `aviso.wav`, os erros sumiram.
- **Bluetooth no emulador.** Demorei um pouco para entender que a busca infinita na tela de pareamento era limitação do emulador e não um problema do código.
- **Achar o app no emulador.** Ao rodar, o relógio ficava no mostrador padrão. Foi preciso abrir a lista de apps (botão 1 da barra do emulador) para encontrar o Doma Assist.
- **Voz no Android 11+.** Para o TextToSpeech funcionar nas versões mais novas é preciso declarar o bloco `<queries>` no manifest.

## 7. Limitações e melhorias futuras

O texto de contextualização fala em ler mensagens e notificações em voz alta e responder a comandos de voz. Essas partes não foram feitas nesta entrega, já que os passos práticos do roteiro se concentram em saídas de áudio, detecção de dispositivos e reprodução. Como melhorias, daria para:

- ler notificações do relógio em voz alta, com um serviço de escuta de notificações;
- aceitar comandos de voz usando o reconhecimento de fala do Android;
- permitir escolher a velocidade e o volume da voz;
- testar em um relógio real com fone Bluetooth para comprovar a detecção dinâmica.

## 8. Conclusão

O trabalho me ajudou a entender como um app de relógio lida com áudio: como descobrir quais saídas existem, como reagir quando um fone entra ou sai e como usar voz sintetizada e arquivos de som. O app final cumpre os passos principais do roteiro e dá uma base boa para uma versão mais completa, com notificações e comandos de voz. Também ficou claro que testar hardware como Bluetooth no emulador tem limites, e que vale testar em um aparelho real quando for possível.

### Repositório

O código-fonte, o README e esta documentação estão no repositório Git entregue ao tutor.